

Giáo Án

Ghi chú cho giảng viên:

- Giảng viên nhớ giới thiệu tổng quan chương trình để học viên biết trong khoá học sẽ học những gì trước khi bắt đầu.

1. Mục Tiêu

- Hiểu **React là gì** và tại sao nên sử dụng React.
- Nắm rõ khái niệm **Virtual DOM** và cơ chế hoạt động.
- Hiểu sự khác biệt giữa **Imperative** và **Declarative** trong lập trình.
- Biết cách **cài đặt môi trường phát triển React** với **Node.js** và **Vite**.
- Hiểu **cấu trúc thư mục** của một dự án React.

2. Nội Dung Bài Giảng

2.1. React là gì? Tại sao nên sử dụng React?

a) React là gì?

- React là một thư viện **JavaScript** để xây dựng giao diện người dùng (UI), đặc biệt phù hợp với **ứng dụng web đơn trang (SPA - Single Page Application)** và các dự án phức tạp.
- Được phát triển bởi **Facebook (Meta)** và hiện nay là một trong những thư viện phổ biến nhất cho lập trình Frontend.

b) Tại sao nên sử dụng React?

- Component-Based**: Xây dựng UI bằng các **thành phần (component)** có thể tái sử dụng.
- Virtual DOM**: Tối ưu hiệu suất bằng cách cập nhật DOM hiệu quả.
- One-Way Data Binding (Ràng buộc dữ liệu 1 chiều)**: Ràng buộc Dữ liệu theo một hướng giúp dễ kiểm soát.
- Hỗ trợ mạnh mẽ từ cộng đồng**: Có nhiều thư viện hỗ trợ và tài nguyên học tập.

c) Ứng dụng của React

- Web App**: Facebook, Instagram, Airbnb, Netflix.
- Mobile App**: Dùng với **React Native** để xây dựng ứng dụng di động trên các hệ điều hành IOS & Android.
- Desktop App**: Dùng với **Electron** để phát triển ứng dụng máy tính (Desktop)

2.2 Giới thiệu về Virtual DOM (DOM Ảo)

Ghi chú cho giảng viên:

- Ví dụ bằng hình ảnh minh họa giúp học viên dễ hình dung hơn (có thể sử dụng các công cụ vẽ như draw.io).

a) Virtual DOM là gì?

- Virtual DOM (V-DOM) là một **bản sao ảo của DOM thật**, giúp tối ưu hóa hiệu suất cập nhật giao diện.

b) Cơ chế hoạt động của Virtual DOM

- React tạo một bản sao **Virtual DOM** khi có thay đổi trong ứng dụng.
- React so sánh **Virtual DOM** mới với **Virtual DOM cũ** (cơ chế *diffing*).
- React cập nhật những phần thay đổi vào **DOM thật** (*reconciliation*).

c) Lợi ích của Virtual DOM

- Tối ưu hiệu suất**: Chỉ cập nhật phần thay đổi thay vì render lại toàn bộ trang.
- Mượt mà hơn**: Giảm số lần thao tác trực tiếp với DOM thật.

- **Dễ bảo trì:** Code rõ ràng, dễ theo dõi và debug.

2.3. Cơ chế Imperative và Declarative

 Ghi chú cho giảng viên:

Giảng viên có thể mô tả dễ hiểu trước:

- **Imperative** giống như một người thợ xây đặt từng viên gạch trực tiếp để xây nhà, tốn rất nhiều thời gian và công sức.
- Còn **Declarative** giống như một người kỹ sư, chỉ cần mô tả trên bản vẽ, sau đó React sẽ thay mặt lập trình viên viết các câu lệnh DOM để cập nhật UI.

a) Lập trình Imperative (Chỉ dẫn từng bước một)

- Lập trình viên điều khiển trực tiếp **từng bước thực hiện** của chương trình bằng code JS thuần.
- Ví dụ cập nhật UI theo cách **Imperative** (Code JS thuần):

```
const button = document.createElement("button");
button.innerText = "Click me";
button.addEventListener("click", () => {
  alert("Hello!");
});
document.body.appendChild(button);
```

- Nhược điểm:
 - Cần theo dõi trạng thái thủ công.
 - Khó bảo trì khi dự án lớn.

b) Lập trình Declarative (Khai báo)

- Lập trình viên chỉ cần mô tả kết quả mong muốn bằng code React. Sau đó React sẽ thay mặt lập trình viên tự động viết các câu lệnh DOM để cập nhật UI.
- Ví dụ trong React (**Declarative**)

```
function App() {
  return <button onClick={() => alert("Hello!")}>Click me</button>;
}
```

- Ưu điểm:
 - Dễ đọc, dễ bảo trì.
 - Hiệu suất tốt hơn nhờ Virtual DOM.

Imperative	Declarative
Chỉ định từng bước cụ thể	Chỉ mô tả trạng thái mong muốn
Khó mở rộng, bảo trì	Dễ mở rộng, dễ hiểu
Cập nhật trực tiếp DOM	React tự động cập nhật DOM (UI)

2.4 Cài đặt môi trường phát triển (Node.js, Vite)

a) Cài đặt Node.js

- Kiểm tra máy học viên đã cài đặt Node.js chưa:

```
node -v
```

- Nếu chưa có, tải tại <https://nodejs.org> và cài đặt.

b) Tạo dự án React với Vite

- Cài đặt Vite CL

```
npm create vite@latest my-app -- --template react
```

- Di chuyển vào thư mục dự án

```
cd my-app
```

- Cài đặt dependencies

```
npm install
```

- Chạy dự án

```
npm run dev
```

- Truy cập **http://localhost:[port]/** để xem kết quả.

5. Giới thiệu cấu trúc folder trong dự án React

Sau khi tạo dự án, cấu trúc thư mục như sau:

```
my-app
|— node_modules/    # Các package cài đặt
|— public/          # Chứa favicon, index.html
|— src/             # Mã nguồn chính của ứng dụng
|   |— assets/      # Chứa ảnh, CSS
|   |— App.jsx      # Component gốc của ứng dụng
|   |— main.jsx     # Điểm bắt đầu của ứng dụng
|— .gitignore       # File bỏ qua khi push lên git
|— index.html       # File HTML chính
|— package.json     # Thông tin dự án
|— vite.config.js   # Cấu hình Vite
```

Trình tự render trong React

1. Trình duyệt tải `index.html`

- Đây là file HTML gốc, trong đó có một thẻ `<div id="root"></div>`, là nơi React sẽ render giao diện.

2. `main.jsx` là điểm khởi động ứng dụng

- Đây là file đầu tiên chạy trong thư mục `src/`.
- Nó tạo root và render component `App` vào thẻ `<div id="root"></div>` trong `index.html`.

Ví dụ trong `main.jsx`:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

3. `App.jsx` là component gốc

- Đây là **component đầu tiên** React render.

- Nó thường chứa các component khác, ví dụ: Header, Footer, Layout,...
- Có thể xem nó như component "chính" của ứng dụng.

Ví dụ trong `App.jsx` :

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <div>
        <a href="https://vite.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>
        <p>
          Edit <code>src/App.jsx</code> and save to test HMR
        </p>
      </div>
      <p className="read-the-docs">
        Click on the Vite and React logos to learn more
      </p>
    </>
  )
}

export default App
```

Giải thích: Component App đơn thuần chỉ là một hàm trả về JSX (giao diện). Nó sẽ được render trong main.jsx.

Tóm lại:

- `index.html` là file HTML chính, chứa một `<div id="root"></div>`.
- `main.jsx` lấy component `App` và render nó vào `#root`.
- `App` là component gốc, nơi chứa toàn bộ giao diện ứng dụng.